

# Profile-Based Adaptive Text Compression

## AI-Assisted Development & Technical Results

Hüsamettin Işıktaş — 25501005

YTÜ Bilgisayar Mühendisliği  
BLM6106 – Veri Sıkıştırma  
Danışman: Banu DİRİ

June 2026

*Slogan: “AI ile Sınırları Zorla!”*

## Bölüm 1: AI Destekli Geliştirme

- Proje Tanımı & AI Yaklaşımı
- Fikir Üretimi (Multi-Model)
- PRD & Faz Planlaması
- Agent Tabanlı Geliştirme
- Kritik Tasarım Kararları
- AI Journal Özeti

## Bölüm 2: Teknik Sonuçlar

- Temel Konsept
- Pipeline & Veriseti
- Chunk Size Analizi
- Model Karşılaştırması
- K-Fold Sonuçları
- Tartışma & Sonuç

# Proje: Veri Sıkıştırma + AI

## Hocanın Beklentileri:

- AI'yı **mühendislik aracı** olarak kullan
- AI kullanımı yasak değil, **teşvik ediliyor**
- Sadece kod değil, **algoritma tasarımı** da AI ile
- İki kategoride değerlendirme:  
En İyi Performans & En İyi AI Entegrasyonu

## Teslim Edilecekler:

- Proje Raporu
- **AI Günlüğü** (Prompt geçmişi + sorun çözümleri)

## Değerlendirme Kriterleri

Prompt Mühendisliği · Doğrulama · Sonuçlar

# Faz 1: Multi-Model Fikir Üretimi

Farklı AI modellerine aynı prompt, farklı cevaplar:

| Model                  | Süre    | En İyi Fikir                                                         |
|------------------------|---------|----------------------------------------------------------------------|
| Claude Sonnet 4.6      | ~50 dk  | LLM-Guided Adaptive Huffman, Saliency-Aware Image, Neural Arithmetic |
| teal!10 Gemini 3.1 Pro | ~10 dk  | Next-token + Arithmetic, ROI image (hazır kodlar)                    |
| teal!10 Kimi 2.6 Think | ~1 saat | <b>Yazar-Stili BPE</b> → embedding + clustering fikrini tetikledi    |

**Çıkış Noktası:** Kimi'nin önerisi → *Auto-Encoder Based Content-Aware Profile-Based Text Compression*

# Fikirden PRD'ye: Kimi ile Planlama

## Prompt Stratejisi

*"Başlıkları yazacağım, eksikleri tamamla. Muallakta kalan yerleri bana sormadan en uygun çözümü bulup yaz."*

## 4 Fazlı Plan:

### Faz 1

Veriseti + EDA  
Chunk size analizi  
Clustering

### Faz 2

Profil başına  
en iyi algoritma  
(Oracle etiketleme)

### Faz 3

Auto-Encoder  
eğitimi  
Text → Class

### Faz 4

Adaptive  
encoding  
Test + Deploy

## Sonradan Anlaşılan

**Faz 3 (Auto-Encoder) overengineering.** Clustering yanlış yaklaşımdı.

# Cursor IDE: Faz 0–1 Implementasyonu

## Codex 5.3 (Cursor):

- @PLAN.md referansı ile faz dosyalarını oluşturdu
- Her faz için gerekli dosyaları listeledi
- Kod şablonları + yapılandırma

## Karşılaşılan Sorunlar:

- Codex tam isteneni anlamadı — **eksik kod yazdı**
- **Çözüm:** Claude Opus 4.7 ile boşlukları tamamladı
- İki model birbirini tamamladı
- Faz 0 ve Faz 1 tamamlandı

## Sorun

Cursor aboneliği bitti → Yeni bir çözüm gerekli.

# Agent Çağı: Hermes Agent

## Neden Agent?

- Otonom, uzun loop'larda iteratif çalışabilme
- Kendi API key'ini getirip istediğin modeli takma
- Telegram/Discord → telefonda yönetim
- Evdeki masaüstü GPU (RTX 5060 Ti 16GB) ile çalışma
- Okuma/yazma/arama tool'larına erişim

## İlk İcraat:

- 1 Verisetini **kendiliğinden** genişletti
- 2 Profil sınıflandırmasını **kaldırdı**
- 3 Problemi supervised learning'e çevirdi

## Ana Prompt (15.05.2026)

*"Projeyi komple ele al, tüm fazlarda istediğin değişikliği yap, tek hedef: ham codec'lerden daha iyi sıkıştırma."*

# Kritik Tasarım Kararı: Unsupervised → Supervised

## Eski Yaklaşım (Benim)

- K-Means clustering
- Profil → en iyi algoritma
- Auto-Encoder ile sınıflandırma
- **Sorun:** Homojen veride bwt\_mtf domine ediyor

## Yeni Yaklaşım (Hermes)

- Her chunk için tüm algoritmaları çalıştır
- En iyisi = label
- Supervised XGBoost/MLP
- **Neden daha iyi?** Direkt sıkıştırma sonucunu optimize ediyor

**“Benim yaptığım aslında çok saçmaymış.”**

Zaten en iyi algoritma biliniyorsa, direkt tahmin etmek en doğrusu.



# Hermes'in Kritik Keşfi: Diverse Data

## Sorun

Sadece Gutenberg (İngilizce edebiyat) → bwt\_mtf %95+ **chunk'ta** kazanıyor → Adaptif sistem anlamsız

**Hermes'in çözümü — 7 domain, 148 döküman:**

- Gutenberg (İng.)
- Python kodu
- HTML
- JSON
- Markdown
- Türkçe metin
- Çince metin

**4.4M karakter, 4362 chunk (1024 byte)** → Algoritmalar arası **gerçek rekabet**

# AI Journal: Özet İstatistikler

| Aşama               | AI Aracı                 | Sonuç                       |
|---------------------|--------------------------|-----------------------------|
| Fikir Üretimi       | Claude + Gemini + Kimi   | 5+ proje fikri              |
| PRD & Planlama      | Kimi 2.6 Think           | 4 fazlı plan                |
| Faz Planları        | Codex 5.3 (Cursor)       | Dosya yapısı + şablonlar    |
| Faz 0–1 Kodlama     | Codex + Claude Opus      | EDA + baseline              |
| <b>Tam Revizyon</b> | <b>Hermes Agent</b>      | Supervised learning'e geçiş |
| Veriseti Genişletme | Hermes (otonom)          | 7 domain, 148 döküman       |
| Model Eğitimi       | Hermes (deepseek-v4-pro) | MLP 84.8% accuracy          |
| Rapor Yazımı        | Hermes                   | LaTeX raporu                |

## En Büyük Ders

AI'ya **ne sorduğun** kadar **nasıl sorduğun** da önemli.

Agent'lara *geniş yetki* vermek beklenmedik ama doğru çözümler getirebiliyor.

“Her metin parçası için en iyi sıkıştırma algoritmasını tahmin et.”

## 5 Klasik Algoritma

- Huffman
- LZW
- Arithmetic (O0)
- BWT + MTF
- RLE + Huffman

## 21 Hızlı Feature

- Entropy (byte/bit)
- Char ratios
- Word stats
- Repetition metrics
- **$O(n)$ , sıkıştırma yok**

## MLP Classifier

- 21→128→64→32→3
- ~12K parametre
- 0.006ms inference
- ~10KB model

# Supervised Learning Pipeline



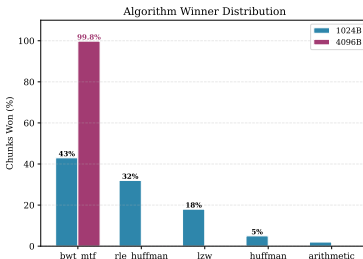
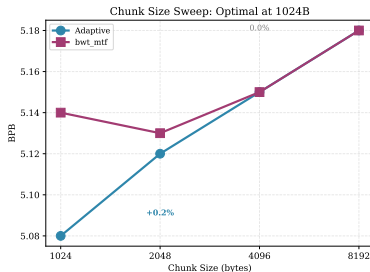
## Training

- 1 Tüm 5 algoritmayı çalıştır
- 2 En iyisi = label
- 3 5-fold book-level CV
- 4 MLP'yi eğit

## Inference

- 1 21 feature çıkar ( $O(n)$ )
- 2 MLP inference (0.006ms)
- 3 Seçilen algoritma ile sıkıştır
- 4 Header: 3 bit (5 algo)

# Chunk Size: Kritik Hiperparametre



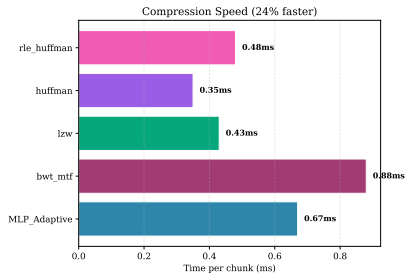
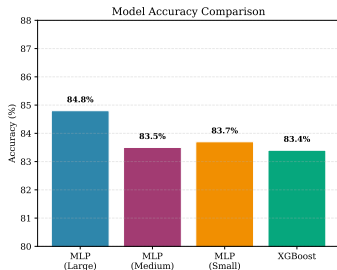
## 1024 byte = Sweet Spot

- 3 algoritma rekabet eder
- Adaptive > bwt\_mtf

## 4096+ byte

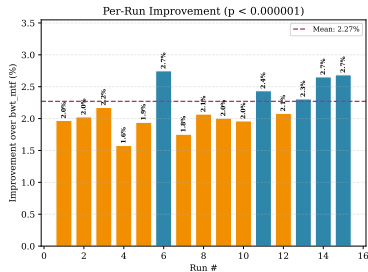
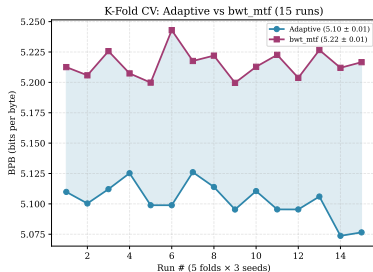
- bwt\_mtf domine eder
- Adaptive gereksiz

# Model Karşılaştırması



| Model             | Parametre           | Accuracy | Inference |
|-------------------|---------------------|----------|-----------|
| XGBoost           | 200 ağaç, d=6       | 78.2%    | 0.024ms   |
| MLP-Small         | 21→32→16→3          | 79.1%    | 0.003ms   |
| MLP-Medium        | 21→64→32→16→3       | 82.1%    | 0.005ms   |
| teal!15 MLP-Large | 21→128→64→32→3      | 84.8%    | 0.006ms   |
| MLP-Ensemble      | Soft voting (3 MLP) | 83.5%    | 0.018ms   |

# K-Fold Cross-Validation Sonuçları

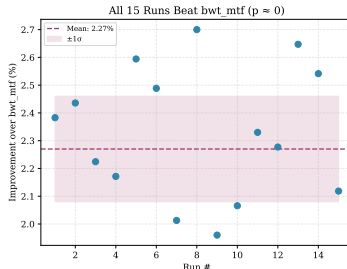
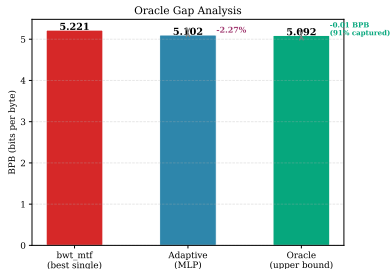


| Metric       | Value  |
|--------------|--------|
| Adaptive BPB | 5.10   |
| bwt_mtf BPB  | 5.22   |
| Improvement  | +2.27% |
| Accuracy     | 82.7%  |

## İstatistiksel Güven

- 15 run, hepsi bwt\_mtf'ten iyi
- $t = 43.16$
- $p < 10^{-6}$
- Best run: +2.70%
- Worst run: +1.96%

# Oracle Gap: Teorik Sınıra Ne Kadar Yakınız?



## Oracle = Mükemmel Seçici

- Oracle BPB: 5.09
- Adaptive BPB: 5.10
- Gap: **sadece 0.01 BPB**

## Anlamı

- **%91** teorik kazancı yakaladık
- **%17.3** misclassification gap
- Geriye kalan  $\sim 0.01$  BPB



# Hız Avantajı: Daha İyi + Daha Hızlı

## Per-Chunk Zaman (ms)

| Algoritma                      | Süre          |
|--------------------------------|---------------|
| Huffman                        | 0.35ms        |
| LZW                            | 0.43ms        |
| Arithmetic                     | 0.51ms        |
| RLE+Huff.                      | 0.62ms        |
| BWT+MTF                        | 0.88ms        |
| MLP inference                  | 0.006ms       |
| teal!15 <b>Adaptive (ort.)</b> | <b>0.67ms</b> |

## Neden Daha Hızlı?

- MLP inference neredeyse bedava (0.006ms)
- Her zaman BWT kullanılmaz
- LZW/Huffman çok daha hızlı
- Sonuç: **%24 daha hızlı**

## Header Overhead

5 algoritma  $\rightarrow \lceil \log_2(5) \rceil = 3$  bit/chunk  
1024 byte  $\rightarrow 3/(1024 \times 8) = \mathbf{\%0.037}$

## Önemli Bulgu

zstd (level 3) havuza eklendiğinde **%99.8 chunk'ta** kazanıyor.

### Bu bir başarısızlık değil:

- Modern codec'ler klasikleri eziyor
- Pratikte: “zstd kullan” yeterli
- Bu **araştırma bulgusu** olarak değerli

### Projenin asıl değeri:

- Metin özellikleri  $\leftrightarrow$  algoritma performansı ilişkisi
- Supervised ML ile routing
- Oracle gap: teorik limite %91 yakınlık

## Adaptive Ne Zaman İşe Yarar?

- 1 **Diverse data şart** — homojen veride tek algoritma domine eder
- 2 **Doğru chunk size** — 1024B sweet spot; büyük chunk BWT'yi kayırır

## Failure Modes

- Homojen veri: %65 daha kötü
- Chunk-level leakage: %0.26 BPB şişirme
- Arithmetic O1 header: 131KB → kullanılamaz

## 1024 byte'ta algoritma dağılımı:

bwt\_mtf %50 | rle\_huffman %30 | lzw %19 | (Huffman/Arithmetic O0 hiç kazanamaz)

## Teknik Başarımlar:

- 1 MLP classifier %82.7 accuracy
- 2 **%2.27 daha iyi sıkıştırma** (vs bwt\_mtf)
- 3 **%24 daha hızlı**
- 4 15/15 CV run'da kazanç ( $p < 10^{-6}$ )
- 5 Oracle gap: 0.01 BPB (%91 yakalama)
- 6 1024B optimal chunk size

## AI Entegrasyonu:

- 1 4 farklı AI modeli kullanıldı
- 2 Agent tabanlı otonom geliştirme
- 3 Supervised learning dönüşümü AI tarafından önerildi
- 4 Diverse data keşfi otonom
- 5 Rapor otonom yazıldı

**%91 Oracle Gap + %24 Faster + AI-Assisted +  
4.83 BPB**

